A one-sided computation guard for iPRA

A smaller proof interface with full-domain bi-interpretations

Research prepared for Vladimir Reshetnikov

November 11, 2026

Abstract

We give a new presentation of intuitionistic primitive recursive arithmetic by changing the guard beneath the paired logical binder. Its atomic matrices have the form $a = I(y, a; u.h)$: the same initializer occurs on the left of equality and inside one iterator. This orientation permits a single introduction schema selected by the matrix root and transport restricted to sources other than the literal term 0. Their compatibility is substantive: the corresponding combination with the previous left-iterator guards loses an equality consequence.

The proof first recovers numerical cases and equation induction, without substituting into a restricted derivation, then restores unrestricted transport and gives both formula composites with the existing native calculus. Composing with the pinned ConciseIPRA bridge gives bi-interpretability with full first-order iPRA, with whole numerical domains, actual equality, and identity numerical composites.

The complete executable checkers and their separately reported size profiles improve the pinned repository frontier under its canonical encoding convention. The reduction is modest; no absolute minimality or new proof-assistant verification is claimed. A strict public-syntax validator, readable checker, reproducible measurements, and positive and negative certificate checks are supplied, with their costs made explicit.

Contents

1 Result and comparison target **4**

2 A mirror guard with one introduction rule **5**
2.1 Complete syntax and primitive rules . . . 5
2.2 Equality operations available before zero-source transport . . . 6
2.3 Guard endpoints, padding, and numerical cases . . . 7
2.4 A small implication sufficient for the bootstrap . . . 8
2.5 Full equation induction without substituting premise proofs . . . 8

3 Why the mirrored guard supports source selection **9**
3.1 A singleton-one test using safe sources . . . 9
3.2 Zero-source symmetry . . . 10
3.3 All equality reasoning and all formula templates . . . 11
3.4 An obstruction for the opposite guard orientation . . . 11

4 Comparison with the native core **12**

5 The ordinary realization and the bridge to iPRA **14**
5.1 The composed interpretations . . . 14
5.2 Why the scalar iterator has the required arithmetic . . . 15
5.3 Exact inherited formal statements . . . 15

6 Complete executable checkers and measured scope **16**
6.1 Canonical statements and administrative proof data . . . 16
6.2 The two dispatch simplifications . . . 17
6.3 Soundness, completeness, and the trusted boundary . . . 17
6.4 Reproduced measurements . . . 18
6.5 Executable evidence . . . 18
6.6 The smallest byte profile in full . . . 19

7 Assessment and limits **19**

A Reference boundary and reproduced arithmetic **20**

B Complementary introductions with one local assumption **20**
B.1 Complete native syntax and rule inventory . . . 20
B.2 Exact equivalence and the unused-witness construction . . . 21
B.3 Induction and numerical cases separate cleanly . . . 22
B.4 The entire ordinary logical interface . . . 23
B.5 Whole-domain realizations and both formula composites . . . 24
B.6 What the theorem does and does not claim . . . 25

C Arithmetic strength and both ordinary interpretation composites **25**
C.1 The arithmetic dependency order . . . 26

D Internal scalar predecessor bootstrap **27**
D.1 Elementary terms and two finite cycles . . . 28
D.2 A ramp with a global scaling identity . . . 28
D.3 A comparator from scalar iteration . . . 29
D.4 Saturated iteration gives predecessor . . . 30
D.5 Dependency boundary . . . 31

E From literal-equation induction to full intuitionistic PRA **31**
E.1 The exact hypotheses . . . 32
E.2 Constructor separation and two elementary tests . . . 32
E.3 Four elementary uses of equation induction . . . 33
E.4 An equation invariant that reflects actual equality . . . 34
E.5 Compilation of all quantifier-free formulas . . . 35
E.6 Recovery of induction and the full theory . . . 36

F Pairing, primitive recursion, and internal numerical composites **37**
F.1 Pairing from a bounded square-root search . . . 37
F.2 Compiling each primitive recursion . . . 38
F.3 Why both numerical round trips are internal . . . 38

1 Result and comparison target

The comparison is with the supplied repository at the exact commit in [1], including the newly formalized no-literal-zero-source theorem. The baseline is therefore not an earlier version that still requires formula transport or witnessed equation progress as primitive. Its current native presentation and its smallest Python profiles are distinguished: the Lean kernel favors an uncomplicated structural metatheory, whereas the compact programs exploit a common arity codec.

The report uses the established clause presentation of the native core: Q_yB is a reusable resource, and its formula representation is $D_{u,y}B$ with unused u. This is connected to the current Lean Eq/Q/D kernel by the inherited equivalence, not a claim that the current kernel lacks Q.

The present change concerns the logical interface, not the numerical domain or the expressive power of the scalar iterator. Keep

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h), \qquad C(v, a, b) := I(v, a; u.b)$$

with u fresh in the selector abbreviation. The previous guarded language allowed an atomic matrix immediately beneath $\mathsf{D}_{x,y}$ only in the form

$$I(y, 0; u.h) = I(y, 0; u.k).$$

Our atomic matrices instead have the form

$$\boxed{a = I(y, a; u.h)}. \tag{M}$$

The initializer a may depend on both x, y and passive parameters; h may also depend on its state variable u. The zero instance is the reverse of one computation axiom. One iterator is displayed rather than two, but no uniform claim is made about expanded term length: the initializer is repeated.

For an arbitrary equation $E \equiv s = t$, the useful particular guard is

$$H_yE \;:=\; s = C(y, s, t).$$

Its zero instance is provable; its successor instances are equivalent to E. The essential detail is that s remains on the left of the equation. In the restricted calculus an assumption $0 = t$ can therefore be padded without first using the forbidden transport from 0. This permits both reductions at once:

1. There is one introduction schema: it retains the progress instance at an equation and the zero instance at a compound matrix. No case/progress choice remains in the certificate.
2. A transport major $s = t$ must have $s \not\equiv 0$ as finite syntax. It is the source field that distinguishes equation transport from closure specialization in the compact certificate protocol.

Both points have predecessors in the supplied repository. The new mathematics is their compatibility with the simpler, oppositely oriented guard and the proof that establishes it without a substitution circularity. A plain combination of the earlier guards and the source restriction would not establish the same theorem.

Meaning of the strength claim. We prove effective translations of all sequents, with formula round trips, between the new presentation $\mathcal{M}$ and the inherited native calculus $\mathcal{C}$. The ordinary reading of every canonical paired formula remains uniform:

$$(\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)\big).$$

After composing with the existing $\mathcal{C} \leftrightarrow T_{\mathrm{eq}} \leftrightarrow$ iPRA bridge, both interpretation domains are defined by $x = x$, equality is actual equality, and the numerical isomorphisms are defined by $x = y$. Thus the result is stronger than agreement on standard numerical computations or an isolated consistency reduction.

Meaning of the size claim. We follow the repository's complete-module convention: remove genuine docstrings, normalize by `ast.unparse`, append one newline, then count significant tokens, AST nodes, and UTF-8 bytes. JSON parsing is the only imported service. Gzip at compression level 9 is reported separately for information. Different complete programs may attain different coordinates; their minima must not be attributed to a single nonexistent program. The strict-goal profile, which also rejects all noncanonical public statements, is measured in full.

Verification boundary. The new metatheorems below are written proofs. The Python checks test the concrete implementations and certificates. The inherited bridge is present in the repository's Lean development [5]; it was inspected but not rebuilt in this environment. No claim of a new Lean or Rocq verification, a universal lower bound, or literature-wide priority is made.

2 A mirror guard with one introduction rule

We combine a restriction of the formula grammar with a restriction of equality transport. The orientation of the guard matters. Its initializer is the *left* side of the guarded equation; the iterator is the right side. This permits the recovery of equation induction before transport from a literal-zero source has been recovered.

2.1 Complete syntax and primitive rules

There is one numerical sort. Numerical terms are

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only the update body h binds u. All other variables occurring in h are fixed numerical parameters. Let

$$1 := S0, \qquad C(v, a, b) := I(v, a; u.b),$$

where the displayed update variable is absent from b. These are abbreviations, not additional primitive symbols.

Canonical formulas are generated by the following clauses.

1. Every numerical equation $s = t$ is a formula.
2. $D_{x,y}B$ is a formula if B is a compound canonical formula.
3. $D_{x,y}B$ is a formula if its literal equation matrix has the form

$$B(x, y) \equiv a(x, y) = I(y, a(x, y); u.h(x, y, u)). \qquad \text{(Mirror matrix)}$$

The bound coordinates x, y are distinct. The finite terms a, h may also contain fixed parameters, and a may depend on y. Bound variables are renamed before substitution. Thus the occurrence of a outside the iterator and its initializer are literally the same term. The guard condition is stable under capture-free substitution for free numerical variables. The designated contradiction is $0 = 1$.

Contexts contain reusable clauses

$$\langle A; y.B(y)\rangle, \qquad y \notin \mathrm{FV}(A),$$

with canonical A, B. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, where v is fresh and $H \equiv I(0,0;u.0) = 0$. Its ordinary hypothesis use is the computation of H followed by the clause-use rule.

The two axiom schemas, in every context, are

$$I(0,a;u.h) = a, \qquad I(Sn,a;u.h) = h[I(n,a;u.h)/u]. \tag{I0,IS}$$

There are precisely four other inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \text{if } \langle A; y.B(y)\rangle \in \Gamma \qquad \text{(Use)},$$

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad s \not\equiv 0 \qquad (\mathrm{Tr}^{-}), \tag{Restricted transport}$$

$$\frac{\Gamma \vdash D_{x,y}B(x,y) \qquad \Gamma, \langle B(b,0); y.B(b,y)\rangle \vdash A}{\Gamma \vdash A} \qquad \text{(DE)},$$

$$\frac{\Gamma, B(t,\eta_B(z)) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta_B(z) = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \qquad \text{(DI)}.$$

(Root-determined introduction)

In transport, F is a canonical numerical formula template, with bound variables fresh for the substituted terms. The restriction $s \not\equiv 0$ is syntactic: variables and all S- or I-headed terms are permitted, including those whose numerical value is zero. In DE, b is fresh for Γ, A and the fixed parameters of B. In DI, z is fresh for Γ, t and the fixed parameters of B. Displayed local clauses and assumptions are discharged.

Call this calculus $\mathcal{M}$. There is no introduction-choice bit, primitive reflexivity, successor axiom, coverage axiom, separation axiom, or additional logical or induction rule. Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. They preserve the syntactic restriction on transport sources. We do *not* assume arbitrary numerical substitution admissible at this stage: replacing a variable source by literal zero need not preserve a primitive transport instance.

2.2 Equality operations available before zero-source transport

Lemma 2.1 (Safe equality operations)**.** *Every term r satisfies $\vdash r = r$. From $p = q$ with $p \not\equiv 0$, one may derive $q = p$ and $K(p) = K(q)$ for any fixed numerical template K. Every computation equation may therefore be reversed as an equation; its reversed direction is usable as a primitive transport major only if its new source is not literally zero.*

Proof. Let $b = I(0,r;u.0)$. The computation $b = r$, used both as transport major and as minor in the template $v = r$, yields $r = r$. The source b has an iterator head. For symmetry, transport $p = p$ through $v = p$ using $p = q$. For numerical congruence, transport $K(p) = K(p)$ through $K(p) = K(v)$. Each source is the given nonzero syntactic term p. Computation equations have an iterator-headed left side, so the symmetry construction applies to them. □

Lemma 2.2 (Joining nonliteral-zero terms at zero)**.** *If $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$, then $\Gamma \vdash p = q$, using computation and restricted transport only.*

Proof. Put $z_0 = I(0, 0; u.0)$ and $b_p = I(0, p; u.0)$. From $b_p = b_p$, transport with $p = 0$ in the template $I(0, v; u.0) = b_p$ to obtain $z_0 = b_p$. Safe symmetry gives $b_p = z_0$. Transport using the computation $b_p = p$ gives $p = z_0$. The identical construction gives $q = z_0$. Safe symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ finishes. All sources used are among p, q, z_0, b_p, b_q, hence are syntactically different from zero. □

Lemma 2.3 (Expansion on the right)**.** *Suppose $q \not\equiv 0$ and $\Gamma \vdash q = r$. Under arbitrary additional assumptions,*

$$s = q \quad \dashv\vdash \quad s = r.$$

In particular, a constructor-headed computation can be expanded or contracted on the right *of an equation without zero-source transport, even when its computed value is literally zero.*

Proof. Contraction uses $q = r$ in the template $s = v$, at the permitted source q. For expansion, suppose $s = r$. If $r \not\equiv 0$, reverse $q = r$ and transport at source r. If $r \equiv 0$ and $s \not\equiv 0$, Lemma 2.2 joins $s = 0$ and $q = 0$. If both r and s are literally zero, safe symmetry of $q = 0$ gives $0 = q$ directly. This is a finite case distinction on term syntax, not an object-level case distinction or substitution into an existing derivation. □

2.3 Guard endpoints, padding, and numerical cases

For an equation $E \equiv (s = t)$ define its selector guard

$$H_v E := s = C(v, s, t). \qquad \text{(Selector guard)}$$

This is the state-independent-update instance of (Mirror matrix) when used under a binder in v.

Lemma 2.4 (Guard endpoints)**.** *For every equation E, uniformly under retained contexts,*

$$\vdash H_0 E, \qquad H_{Sw} E \dashv\vdash E.$$

Every canonical atomic matrix $B(x, y)$ also has the theorem $B(t, 0)$ for every witness term t.

Proof. The base is the symmetry of the single zero computation $C(0, s, t) = s$. For the successor endpoint, let $q = C(Sw, s, t)$. The computation $q = t$ and Lemma 2.3 give exactly the displayed interderivability. For a general canonical atomic matrix, its zero instance is $a(t, 0) = I(0, a(t, 0); u.h(t, 0, u))$, again the symmetry of one zero computation. No congruence or equality-reflection theorem is assumed. □

Lemma 2.5 (Case introduction at canonical matrices)**.** *The case form of DI is admissible for every canonical matrix, including literal equations.*

Proof. For a compound matrix it is primitive. For an atomic canonical matrix B, cut its theorem $B(t, 0)$ into the given case premise $\Gamma, B(t, 0) \vdash B(t, Sz)$. Weaken the result by $B(t, z)$ and apply the primitive progressive introduction. This establishes only case introduction for matrices allowed by the canonical grammar. □

For an equation E put

$$L(E) := D_{a,v} H_v E, \qquad P(A) := \begin{cases} L(A), & A \text{ is an equation,} \\ A, & A \text{ is compound.} \end{cases} \qquad \text{(Padding)}$$

The new bound variables are fresh, and a is unused. In particular, $P(A)$ is always compound.

Lemma 2.6 (Padding)**.** *$L(E) \dashv\vdash E$ and $P(A) \dashv\vdash A$ under arbitrary retained contexts.*

Proof. Under E, use guard introduction to prove $H_{Sz}E$ while ignoring the local induction assumption H_zE. Atomic DI with witness zero gives $L(E)$. Conversely, DE opens its clause $\langle H_0E; v.H_vE\rangle$. The base is a theorem. Use at 1 and successor guard removal give E. No numerical substitution into an arbitrary premise derivation occurs. □

Lemma 2.7 (Numerical cases)**.** *For any canonical formula $A(y)$, the rule*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)} \tag{NC}$$

is admissible, provided z is fresh for Γ, t and the fixed parameters of A.

Proof. If A is compound, introduce $D_{u,y}A(y)$ by case DI with unused u and witness zero, ignoring its local base assumption. DE opens $\langle A(0); y.A(y)\rangle$; the given base proof and Use at t finish. If A is an equation, pad the two premises, apply the compound case rule to $L(A(y))$, and unpad its conclusion. The retained context is unchanged throughout. In particular, this is not a rule permitting case substitution in assumptions that contain the case variable. □

2.4 A small implication sufficient for the bootstrap

We need implication only between equations with a shared left side. For fixed terms s, a, b, let

$$M_w(s; a, b) := s = C(w, a, b), \qquad \mathrm{Imp}_s(a, b) := D_{u,w}L(M_w(s; a, b)), \tag{Shared-left implication}$$

with fresh controls and unused u. Lemma 2.3 applied to the zero and successor computations yields

$$M_0(s; a, b) \dashv\vdash (s = a), \qquad M_{Sz}(s; a, b) \dashv\vdash (s = b). \tag{Shared-left endpoints}$$

The zero cases of the right endpoints are covered by zero-join. No double-selector conversion that changes both sides of an equation is being used.

Lemma 2.8 (Rules for shared-left implication)**.** *The following are admissible:*

$$\frac{\Gamma, s = a \vdash s = b}{\Gamma \vdash \mathrm{Imp}_s(a, b)}, \qquad \frac{\Gamma \vdash \mathrm{Imp}_s(a, b) \qquad \Gamma \vdash s = a}{\Gamma \vdash s = b}.$$

Proof. For introduction, the outer matrix $L(M_w)$ is compound. Its case assumption $L(M_0)$ unpads and contracts to $s = a$. Apply the supplied derivation, expand $s = b$ to M_{Sz} using (Shared-left endpoints), and pad. Compound DI closes. For elimination, DE opens the outer clause. Convert $s = a$ to its padded zero antecedent, use the clause at 1, unpad, and contract to $s = b$. The supplied derivation is used at its original parameters; only fresh control variables are introduced. □

2.5 Full equation induction without substituting premise proofs

Theorem 2.9 (Contextual equation induction)**.** *For every literal equation $E(y)$, $\mathcal{M}$ admits*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \tag{EI}$$

with the usual fresh-parameter condition on z. No transport from a literal-zero source or numerical substitution into either supplied premise derivation is used in its recovery.

Proof. Write $E(y) \equiv s(y) = r(y)$ and put

$$J(y) := s(y) = C(y, s(y), r(y)),$$

$$\Phi(y) := \mathrm{Imp}_{s(y)}(C(y, s(y), r(y)), r(y)). \qquad \text{(Bootstrap implication)}$$

Thus $\Phi(y)$ has the derived introduction and elimination rules of $J(y) \to E(y)$. Its root is compound.

Under Γ, prove $\Phi(0)$ by shared-left implication introduction, ignoring the assumption $J(0)$ and using the supplied base proof $E(0)$. For a fresh w, prove $\Phi(Sw)$ by the same introduction: its local hypothesis $J(Sw)$ contracts by the successor guard endpoint to $E(Sw)$. These proofs retain only Γ, independent of the case variable. NC therefore proves $\Phi(v)$ for every target term v.

In particular, under the new local assumption $J(z)$, the already available $\Phi(z)$ gives $E(z)$. Apply the *original* supplied step derivation at this same eigenparameter z, obtaining $E(Sz)$. The successor guard endpoint yields $J(Sz)$. Since $J(y)$ has the canonical mirror shape, atomic progressive DI with an unused witness coordinate gives $D_{u,y}J(y)$.

Eliminate this binder. Its clause has the theorem $J(0)$ as antecedent, so Use at the desired target t supplies $J(t)$. The independent NC instance $\Phi(t)$ then gives $E(t)$ by shared-left implication elimination. Close DE.

Every invocation of a guard or selector endpoint is a direct instance of its uniformly constructed proof. In particular, the step derivation has not been specialized at 0, at Sw, or at any other new term. This is the point that avoids circular reliance on unrestricted numerical substitution in a source-restricted calculus. □

3 Why the mirrored guard supports source selection

The source restriction is syntactic: a permitted transport source is any term other than the single literal 0. In particular, a variable or an I-headed term whose numerical value is zero is permitted. The numerical cases and equation induction established above were proved under precisely this restriction. Their derivations did not substitute into an existing proof at a variable source. This point makes the following transfer noncircular.

Theorem 3.1 (Restoration of zero-source transport)**.** *The mirrored-guard calculus with transport restricted to sources $s \not\equiv 0$ derives exactly the same sequents as its version with unrestricted transport. The formula language, contexts, computations, and introduction and elimination rules are unchanged.*

We supply the zero-source construction, adapting the argument of the pinned `source_dispatch` package [3]. Its application here is new: the unrestricted atomic case/progress language of that package has been replaced by the mirrored guarded language. Lemmas 2.7 and 2.9 supply the numerical rules that the construction requires.

3.1 A singleton-one test using safe sources

Use the abbreviations

$$\begin{array}{ccc} C(v,a,b) = I(v,a;u.b), & 1 = S0, & 2 = S1, \\ Z(v) = C(v,1,0), & K(v) = C(v,0,1), & H(v) = C(v,1,2), \\ N(n) = I(n,0;u.Z(u)), & R(n) = I(n,0;u.H(u)), & \ell(n) = N(R(n)). \end{array}$$

Every displayed update variable is fresh for the fixed parameters. In this subsection all case distinctions are performed in the empty retained context. The results may subsequently be weakened into a context containing a zero-source equation.

Lemma 3.2 (Uniform test and filter equations)**.** *At every displayed target, the restricted calculus proves*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$
$$N(1) = 1, \qquad N(2) = 0, \qquad \ell(Sv) = Z(v),$$

and

$$v = C(Z(v), v, 0). \tag{Filter}$$

These are uniform derived rules at arbitrary terms, without an assumption that numerical substitution into restricted derivations is admissible.

Proof. For $H(v) = SK(v)$ use numerical cases. At zero both sides compute to 1; at a successor both compute to 2. A computation may be reversed as an equation because its original left side is I-headed. Joining the resulting equations through 1 or 2 uses a permitted numeral source. Thus this case argument has no zero-source transport.

Finite forward computations give $R(1) = 1$, $N(1) = 1$, and $N(2) = 0$. For example, $N(1) = Z(N(0)) = Z(0) = 1$, and the next iterate gives $N(2) = Z(1) = 0$. Every replaced subterm is the left side of a computation or an already derived I-headed equation.

The first identity and the successor computation give

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Replace the I-headed subterm $R(Sv)$ in $R(SSv) = H(R(Sv))$. The result computes to 2. Hence $\ell(1) = 1$ and $\ell(SSv) = 0$ by the computed values of N. Numerical cases now give $\ell(Sv) = Z(v)$: the zero case joins the two values 1, while the successor case joins the two I-headed terms known to be zero using Lemma 2.2.

For the filter equation use numerical cases on its displayed variable. At zero, the right side computes forwards to zero; at a successor it computes forwards to that successor. Reverse each resulting equation at its I-headed source. This proves the required endpoints and then the identity at any target. The argument uses no retained assumption depending on the case variable. □

3.2 Zero-source symmetry

Lemma 3.3. *For every numerical term t, the restricted calculus derives*

$$0 = t \;\vdash\; t = 0.$$

Proof. If t is literally zero, use reflexivity. Otherwise retain $0 = t$, keep t fixed, and abbreviate

$$A(n) = I(n, t; u.Su).$$

We prove the equation $n = A(n)$ by the already established equation induction rule, Lemma 2.9.

At zero, $A(0) = t$ is a computation. Its safe symmetry gives $t = A(0)$. Transport the retained $0 = t$ at the permitted source t to obtain $0 = A(0)$.

For the step, retain $z = A(z)$ at the same fresh parameter z. Its source is the variable z, hence permitted. Safe successor congruence gives $Sz = SA(z)$. Computation and safe symmetry give $SA(z) = A(Sz)$; transport at the successor-headed source $SA(z)$ gives $Sz = A(Sz)$. In particular, this step proof is used at its original parameter: it is never specialized to zero.

Induction at target 1 yields $1 = A(1)$. Forward computations give $A(1) = St$, so transport at source $A(1)$ yields $1 = St$. Its source is the permitted numeral 1. Congruence for ℓ yields

$\ell(1) = \ell(St)$. Replace the I-headed terms using $\ell(1) = 1$ and $\ell(St) = Z(t)$, obtaining $1 = Z(t)$, and safely reverse this equation to obtain $Z(t) = 1$.

Weaken the unconditional filter equation (Filter) into the retained context. Transport at the I-headed source $Z(t)$ turns its right side into $C(1, t, 0)$. This term computes to zero, again at an I-headed source. The result is $t = 0$. □

3.3 All equality reasoning and all formula templates

Lemma 3.4 (Unrestricted numerical equality algebra)**.** *The restricted calculus admits symmetry, transitivity, and numerical congruence for all equations, without a source restriction.*

Proof. Symmetry at a nonzero syntactic source was already available; at zero use Lemma 3.3. For congruence from $0 = t$ with $t \not\equiv 0$, first reverse it, apply permitted congruence at source t, and use the now unrestricted symmetry. The case $t \equiv 0$ is reflexivity.

Transitivity through a middle term other than literal zero uses one permitted transport. For $r = 0$ and $0 = t$, either an endpoint is literally zero and a premise is already the conclusion, or both endpoints are nonzero syntactic terms. In the latter case reverse $0 = t$ and apply the safe zero-join lemma to $r = 0$ and $t = 0$. None of these arguments uses the unrestricted rule being derived. □

Lemma 3.5 (Unrestricted canonical-template transport)**.** *For every canonical formula template $F(v)$,*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in the source-restricted mirrored calculus.

Proof. Induct externally on the number of binders in the finite template, uniformly in contexts and fixed parameters. For an equation use Lemma 3.4: congruence on its two terms, symmetry, and transitivity give the desired replacement.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh witness a, retaining its clause $Q_yB(s, a, y)$, where Q_yC here abbreviates the reusable clause $\langle C(0); y.C(y)\rangle$ rather than a new object-formula constructor. To introduce $F(t)$, assume $B(t, a, 0)$ and seek $B(t, a, Sz)$. The induction hypothesis, using symmetry of $s = t$, gives $B(s, a, 0)$. Use the old clause at Sz and transport forwards by the induction hypothesis to obtain $B(t, a, Sz)$.

For a compound matrix this is the primitive case introduction. For an atomic canonical matrix it is a derived case introduction: its zero instance is a theorem by safe symmetry of I0; cut that theorem into the displayed case premise, ignore the local progress hypothesis, and apply atomic progress introduction. Thus the construction is valid for every canonical matrix. Close the introduction and the elimination. All eigenparameters are fresh for the equality and retained data. □

Proof of Theorem 3.1. Each restricted proof is an unrestricted proof. Conversely, transform an unrestricted proof recursively and replace each transport node by Lemma 3.5. Other inference nodes are unchanged. The contexts and conclusions are identical. This proves equality of the entire consequence relations, not only of their closed theorems. □

3.4 An obstruction for the opposite guard orientation

The direction of the guard matters before zero-source transport has been restored. The following statement explains why it is not enough to combine the two older restrictions without changing the guard.

Consider a free-syntax calculus whose object formulas are equations and D formulas. Admit a D formula with atomic matrix only when the left side of that matrix has a fixed constructor head which survives every capture-free numerical substitution. For example, this applies to

$$I(y, 0; u.h) = I(y, 0; u.k) \quad \text{and} \quad C(y, t, s) = t.$$

Use the same computations, source-restricted transport, DE/Use, compound case introduction, and atomic progress introduction as above. Ordinary hypotheses are retained formulas. Clauses introduced by DE have the corresponding admissible matrix. No arbitrary additional clause is present in the statement below.

Call a formula *safe* when it is a D formula, or an equation whose left side is not literally zero. Call a retained clause safe when its antecedent and all its numerical instances are safe.

Proposition 3.6 (Inert zero-left hypothesis)**.** *In this left-constructor-guard calculus, if a safe formula F is derivable from a context of safe hypotheses and safe clauses together with one extra hypothesis $0 = t$, then F is derivable from the same context with that extra hypothesis erased.*

Proof. Induct on a derivation whose conclusion is safe. A safe hypothesis cannot be the erased zero-left equation. Computation conclusions are safe. Atomic progress introduces a local hypothesis and has a premise with the fixed constructor head on the left; these are safe. Compound case introduction has a compound local hypothesis and premise. In DE the major formula is compound, and its new clause is safe because its matrix is either compound or has the stipulated left constructor head. Use on a safe clause has a safe antecedent, to which the induction hypothesis applies. Thus all these cases admit erasure recursively.

For transport, its major $p = q$ is safe because $p \not\equiv 0$. If the conclusion $A(q)$ is safe, then its minor $A(p)$ is also safe. This is immediate for a D root. For an equation, a numerical template whose left instance at the nonzero syntactic term p is literally 0 must have constant left side 0: neither a constructor-headed term nor an occurrence of the hole can become the free-syntax literal zero when the hole is filled by p. Its instance at q would then also have left side 0, contrary to safety. The induction hypothesis therefore applies to both transport premises. This proves erasure. □

For the two concrete guarded shapes displayed above, the calculus is sound in the standard natural numbers with the ordinary meanings of the binders. Consequently it cannot prove $1 = 0$ without assumptions. Proposition 3.6 therefore implies

$$0 = 1 \nvdash 1 = 0.$$

Thus these source-restricted presentations fail to recover even this instance of ordinary equality symmetry or explosion. The proposition does not apply to arbitrary externally supplied clauses with a zero-left antecedent and a safe consequent, to definitional quotients of term syntax, or to other additional inference rules. Those would change its hypotheses. It applies to the proposed direct combination of the original canonical guarded language and the source restriction.

The mirrored guard escapes the obstruction because its left term may be the induction variable itself. The bootstrap through right expansion, shared-left implication, and equation induction makes that extra possibility effective.

4 Comparison with the native core

The safe equality operations, NC, and EI are now available with their full contextual force. The zero-source restoration argument in Theorem 3.1 therefore applies: it derives $0 = t \vdash t = 0$,

full numerical equality algebra, and unrestricted canonical formula transport. Its only nontrivial induction uses the equation $n = I(n, t; u.Su)$ with t fixed; Theorem 2.9 supplies precisely that rule without a forbidden proof substitution. Case DI for every canonical matrix was already obtained in Lemma 2.5, so the binder step in the restoration of general formula transport is legitimate.

Let $\mathcal{C}$ be the inherited Choice-DI calculus: the same terms, all formulas generated freely from equality and D, the same computation, Use and DE rules, unrestricted transport, case DI at every matrix, and progressive DI at literal equations. Define

$$T(s = t) := (s = t), \qquad T(D_{x,y}B) := D_{x,y}P(TB), \tag{Translation}$$

renaming all padding variables freshly. Translate a context clause by $T\langle A; y.B\rangle := \langle TA; y.TB\rangle$. Let $J : \mathcal{M} \to \mathcal{C}$ be literal inclusion of the canonical syntax. Neither translation changes a numerical term.

Theorem 4.1 (Translation of derivations)**.** *The following effective transformations preserve the full consequence relations:*

$$\Gamma \vdash_{\mathcal{C}} A \Longrightarrow T\Gamma \vdash_{\mathcal{M}} TA, \qquad \Gamma \vdash_{\mathcal{M}} B \Longrightarrow J\Gamma \vdash_{\mathcal{C}} JB.$$

Proof. Inclusion preserves every primitive rule: source-restricted transport is an unrestricted instance, and root-directed DI chooses an available case or progress rule. For T, induct on a $\mathcal{C}$ derivation. Computation is unchanged. Use commutes with formula substitution. Transport uses the translated template and restored unrestricted transport.

A case-DI step is translated by introducing the compound matrix $P(TB)$. Unpad its local base, apply the translated premise, and pad the successor result. For a progressive step at an equation E, the target is $D_{x,y}L(E)$. Introduce it by compound case DI. Rename the supplied step eigenparameter freshly, away from the new outer successor target, before applying EI. Its base $L(E(t, 0))$ supplies $E(t, 0)$. Recovered EI applied to the translated step then gives $E(t, Sz)$, which padding converts into the required successor target. Before this EI application, rename the step's eigenparameter to a fresh w, distinct from the outer case parameter z occurring in the target Sz. This is a safe renaming, not substitution of a numerical term into a derivation.

For DE, the translated major opens the clause $\langle P(TB)(b, 0); y.P(TB)(b, y)\rangle$. The translated minor expects $\langle TB(b, 0); y.TB(b, y)\rangle$. Replace each use of that expected clause by padding its antecedent, using the available clause at the same target, and unpadding its consequent. This local replacement preserves the fresh-witness condition. All other cases are immediate. □

Lemma 4.2 (Canonical atomic composite)**.** *For every admissible mirror matrix $B(x, y)$,*

$$D_{x,y}B \quad \dashv\vdash_{\mathcal{M}} \quad D_{x,y}L(B).$$

Proof. Forwards, open the source clause at a fresh witness b and introduce the compound target matrix by case DI. Unpad its base, use the source clause at Sz, and pad its successor instance.

Backwards, open the clause of $D_{x,y}L(B)$ at b. The mirror zero-instance theorem proves $B(b, 0)$, hence $L(B(b, 0))$. The opened clause yields $L(B(b, Sz))$, and unpadding gives $B(b, Sz)$ independently of the local hypothesis $B(b, z)$. Atomic progressive DI therefore introduces $D_{x,y}B$ at witness b. Close DE. □

Theorem 4.3 (Both composites and the whole-domain interpretation)**.** *For every $\mathcal{C}$-formula A and canonical $\mathcal{M}$-formula B,*

$$A \dashv\vdash_{\mathcal{C}} JT(A), \qquad B \dashv\vdash_{\mathcal{M}} TJ(B).$$

The ordinary realization of $\mathcal{M}$ is bi-interpretable and equiconsistent with iPRA through the inherited native-core bridge. Both numerical domains are whole, numerical equality remains actual equality, and both numerical composites have identity graph $x = y$.

Proof. Padding is interderivable with its argument in $\mathcal{C}$, and case DI gives congruence under D. Structural induction therefore gives $A \dashv\vdash JT(A)$.

In $\mathcal{M}$, uniformly interderivable compound matrices may be exchanged under D: eliminate the source binder, introduce the target by compound case DI, convert its base to the source base, use the opened clause at a successor, and convert back. The reverse direction is identical. Structural induction on a canonical formula now proves the second composite. At a compound matrix use this congruence; at an atomic mirror matrix use Lemma 4.2. The transformations are uniform in retained contexts and therefore extend to clauses by replacing their Use instances.

The ordinary reading leaves equations and terms unchanged and reads

$$D_{x,y}B \quad \text{as} \quad \exists x(B(x,0) \to \forall y\, B(x,y)),$$

and $\langle A; y.B\rangle$ as $A \to \forall y B$. The numerical interpretations in the two displayed native translations are literal identities. Composing them, and their proved formula composites, with the native-core/ordinary-scalar/iPRA interpretations gives the claimed whole-domain bi-interpretation. The contradiction $0 = 1$ is unchanged, so equiconsistency follows. No standard-numeral restriction, quotient equality, or uniform evaluator for function descriptions is introduced. □

Scope of the reduction. The new primitive inventory uses one root-determined introduction and omits all literal-zero-source transports. Its guard uses a single iterator and repeats its initializer, on the opposite side of the equation. The proofs above establish preservation of the full consequence relation after the displayed formula translations. They do not establish global minimality of an axiomatization or a shortest possible implementation. A raw certificate checker that accepts noncanonical goals must additionally state its administrative reading; the theorem about canonical formulas alone is not a theorem that such a program validates the canonical grammar.

5 The ordinary realization and the bridge to iPRA

Here "iPRA" means full intuitionistic first-order logic with actual numerical equality, a function symbol for every fixed finite primitive recursive description, the defining equations, successor separation and injectivity, and induction for quantifier-free formulas with arbitrary retained assumptions. It does not mean just its equational fragment, and no excluded-middle principle for quantified formulas is added.

The existing scalar realization T_{eq} uses ordinary intuitionistic logic on $0, S, I$, the two iterator equations, the coverage axiom

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

the falsity bridge $(0 = S0) \to \bot$, and contextual induction for literal equations. A fixed update term is elaborated into a fixed function description. There is no numerical variable ranging over programs and no assumed universal primitive recursive evaluator.

5.1 The composed interpretations

Let $j : \mathcal{M} \to \mathcal{C}$ be inclusion of the canonical numerical and formula syntax, and let $T : \mathcal{C} \to \mathcal{M}$ be the padding translation proved above. Let R and U be the inherited interpretations from $\mathcal{C}$ to iPRA

and back. The new pair is

$$Rj : \mathcal{M} \longrightarrow \text{iPRA}, \qquad TU : \text{iPRA} \longrightarrow \mathcal{M}.$$

The preceding proofs give $TjB \dashv\vdash B$ and $jTA \dashv\vdash A$. The inherited formula composites give $URA \dashv\vdash A$ and $RU\varphi \leftrightarrow \varphi$. Applying the faithful translations and composing these equivalences yields both new formula composites. This records the actual bi-interpretation argument; mutual interpretability alone would not have been enough.

The numerical maps between $\mathcal{M}$ and $\mathcal{C}$ are identities. Between T_{eq} and iPRA they replace each fixed function description by a finite scalar term or vice versa. Their internal round trips are proved by uniqueness of primitive recursion. The same whole numerical domains and identity graphs therefore serve for the new composites. Native $0 = S0$ translates to ordinary falsity in both directions, so finite proof translations give equiconsistency.

5.2 Why the scalar iterator has the required arithmetic

The numerical argument is inherited, not a new assumption hidden in the proposed proof checker. The complete written sources are reproduced in the appendices. Their dependency order is as follows.

1. Literal-equation induction and numerical cases construct a scalar predecessor P with $P0 = 0$ and $P(Sn) = n$. This is the part that cannot be replaced by a casual assertion that an iterator can simulate arbitrary recursion: the update has only one scalar state and has no recursion-counter binder.

2. Predecessor gives successor injectivity. Selector computations and separation give constructor disjointness. Truncated subtraction and an equality test are then derived and proved to reflect actual equality throughout the domain.

3. Characteristic terms for quantifier-free formulas yield full quantifier-free induction, including its intuitionistic propositional connectives. The equality test is proved internally before it is used.

4. Pairing and its two projections are constructed using bounded scalar iteration. General primitive recursion is compiled into an iterator storing both its counter and its previous value.

For example, with a proved pairing π and projections L, H, a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p})$$

is represented by

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$, with all parameters fixed. The two projection identities then prove both defining equations for f^*. Uniqueness of the two recursive constructions proves the internal function composites, including at nonstandard elements of models; standard-model extensional agreement is not being substituted for that argument.

5.3 Exact inherited formal statements

The pinned file `NativeBiInterpretation.lean` contains `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`. The scalar bridge is

`biInterp : BiInterp TeqAx PRAAx` in `BiInterpretation.lean`. The reports' rule presentation is connected to that native kernel by `iprf_iff_nprf`. These are the inherited results used here, rather than a claim that the new mirror restriction has already been inserted into Lean.

A Lean continuation would introduce the mirror-guard predicate, the root-determined proof relation, and the two syntax translations, then formalize safe right expansion, shared-left implication, the substitution-free induction construction, and the formula composites. It should not start by restricting the existing transport constructor and invoking its structural substitution theorem: a source variable can become the forbidden literal 0 under such a substitution. After full transport has been recovered, unrestricted substitution is admissible through the established equivalence.

6 Complete executable checkers and measured scope

6.1 Canonical statements and administrative proof data

The public mathematical language is precisely the canonical language of Section 2. The small programs use the repository's arity encoding of that language. The same raw arrays also provide administrative resources for proof construction, including closure clauses and harmless zero or truth aliases. This is a specified decoding contract; the compact profiles do not reject every noncanonical end statement. The separate `code/strict_checker.py` does validate the public goal before checking its proof, and its complete cost is reported separately.

Input is one JSON array. The last item is the goal; preceding items are instruction packets, consumed from right to left. The sole successful outcome is normal termination with exactly `ACCEPT` and a newline. An exception, a nonzero exit, a timeout, or resource exhaustion is not acceptance. Unused prefix data is inert, as in the pinned protocol. There is no imported producer, normalizer, axiom database, or metatheory package in a checker.

The raw numerical and formula constructors are:

Array length	Numerical position	Formula position
0	zero	a fixed true equation
1	successor	paired binder D
2	zero annotation	equality
3	iterator	administrative closure Q
≥ 4	zero annotation	truth annotation

A canonical closure resource has three copies of its matrix. A unary major $[B]$ exports $[B, B, B]$ when its witness is opened; tripling any other nonempty formula arity creates only a truth annotation. In the canonical numerical syntax, literal 0 is the integer zero and $S0$ is `[0]`.

Protected numerical slots have width two. Numerical root depth is 3, formula root depth is 0, and traversal increases depth by $5 -$ array length. The coordinates $0, 1, 2$ denote zero; the odd coordinate $3 + 2j$ canonically names protected slot j. Negative coordinates name rigid parameters allowed by the current freshness floor. Successors introduce two dummy slots; an iterator introduces one slot, dummy in its counter and initializer and genuine state in its update. The inherited sorted-opening and freshness argument is retained in `reference/inherited_arity_checker.tex`.

The strict validator checks the mirrored initializer after raising the left-hand numerical term across the iterator's additional slot. Comparing the two raw initializer arrays directly would incorrectly reject genuine bound-variable occurrences, including nonvacuous witnesses. Its goal traversal and raising routine are included in the measured module.

6.2 The two dispatch simplifications

In a lexical-scope packet `[m,s,u,t]`, the first field is a major premise or the neutral truth marker. Source selection is

```
m == ([s, u] if s else [t] * 3)
```

For a canonical source other than literal zero, the major must be the equation $s = u$. At a literal-zero source it must be the closure resource with body t. The theorem restoring zero-source transport justifies this loss of a primitive equation-major alternative.

The retained local assumption for an introduction uses

```
(m or not len(a) % 2) and k
```

instead of the preceding protocol's comparison with a certificate mode field. A meaningful even formula root is an equation and receives the fresh progress parameter. A meaningful odd root is compound and receives zero. The mirrored grammar and its bi-interpretation justify deleting this independent choice.

Computation packets construct a zero or successor instance of the iterator equations. The packet format requires a nonempty initializer and, for a successor computation, a nonempty predecessor. These restrictions are discharged by the inherited certified zero

$$c_0 = I(2, 1; u.0), \qquad c_0 = 0.$$

Its equation is directly admitted. Replace the offending literal-zero positions of a desired computation by c_0, produce the admitted equation, then transport using $c_0 = 0$. This uses an iterator-headed source and so remains available before zero-source restoration. It changes the proof, not the literal zero or initializer occurring in a public mirror matrix.

6.3 Soundness, completeness, and the trusted boundary

Theorem 6.1 (Canonical checker theorem)**.** *For each closed canonical $\mathcal{M}$ formula A, there exists an accepting finite certificate with encoded goal A exactly when $\mathcal{M} \vdash A$, in the ideal unbounded execution model. The strict checker additionally rejects noncanonical public goals.*

Proof. For soundness, interpret the administrative formulas by the ordinary anchored reading of the inherited native calculus: $Q_y B$ means $B(0) \to \forall y B(y)$, and $D_{x,y} B$ means its existential closure. An atomic progress introduction is sound for this reading by literal-equation induction; a compound case introduction is sound by numerical coverage. Thus no progressive interpretation of a raw unguarded binder, and no formula-composite claim for arbitrary raw arrays, is needed.

Induct on a finite successful call tree. The inherited opening lemma verifies numerical substitution, dummy slots, and fresh parameters. The major is checked before its result contributes any proof resource. Source-selected transport and closure use are instances of the inherited rules. A unary major supplies its correct clause; other exported arities are truth. Constructed computations are discharged by the inherited computation-supply argument. The root test admits progressive hypotheses only at literal equations or truth annotations; all compound matrices receive their zero instance. These are sound inherited proof steps. Hence every successful canonical end statement has a proof of its ordinary reading in iPRA. The established faithfulness and round trips translate that proof back to $\mathcal{M}$.

For completeness, encode a finite $\mathcal{M}$ derivation with fresh integer parameters. Use the inherited packets for clause use, transport, and paired elimination. Every permitted source is nonempty in canonical encoding, so its equation major is selected. Every atomic DI is progressive and every

compound DI is by cases, exactly as the root test requires. Certified-zero expansion supplies all computation instances. This produces a finite accepting certificate. The strict validator accepts the stipulated goal encoding, so it has the same canonical completeness. □

The program may accept an unguarded administrative goal whose ordinary reading is provable. It is not asserted to be complete for all such goals, nor are they new object-language formulas of $\mathcal{M}$. The strict profile makes this public boundary executable. A comparison requiring rejection of every out-of-language public input must use the strict rows, not the compact rows alone.

The AST profile uses lexical scopes and an explicit loop; the byte profile uses a recursive continuation with the same lexical scopes. The token profile uses the inherited global supply of fresh parameter names. Certificates for the global profile are emitted separately. All three are complete standalone programs rather than partial fragments. A readable independent expansion is also included.

6.4 Reproduced measurements

Complete program	Tokens	AST nodes	Bytes	Gzip-9
Source dispatch: tokens	259	289	572	339
Mirror: tokens	258	287	571	339
Source dispatch: AST	263	283	607	348
Mirror: AST	262	281	606	348
Source dispatch: bytes	266	305	526	330
Mirror: bytes	265	303	525	331
Mirror: readable	279	319	1272	503
Source: strict derivative	471	537	1127	560
Mirror: strict	578	661	1503	673

All byte and gzip columns use normalized UTF-8 source. The JSON evidence also records raw-source sizes, raw gzip sizes, hashes, and exact runtime versions. The strict source derivative is constructed for this comparison; it is not an upstream frontier claim.

The comparison files are frozen verbatim from the pinned `source_dispatch` package and retain their hashes. The baseline coordinate minima are 259 significant tokens, 283 AST nodes, and 526 normalized bytes; they belong to three different programs. The supplementary gzip measurements use level 9 and timestamp zero. They are measurements, not an objective used to justify simplicity. The main reduction is small, and the strict syntax-validation cost does not disappear merely because the compact codec permits a canonical-encoding contract.

6.5 Executable evidence

The reproduction run executed 480 checker invocations on 48 distinct fixtures; 480 outcomes matched their declared expectations. Every fixture was checked in both ordinary and Python optimization mode on each of the five implementations.

There are 15 positive named proof trees. Their structural audit found no nonmirrored atomic introduction and no primitive transport with literal-zero source.

The zero-source normalization example has 471 distinct named nodes, 13 atomic introductions, and 9 compound introductions. It has 507 certificate instructions. Its local and global certificates are retained in the archive.

A second closed certificate proves the shared-left implication $(0 = 1) \Rightarrow (0 = 2)$, using the variable-left invariant $n = I(n, 1; u.Su)$. It has 279 distinct named nodes and 266 instructions, with 8 mirrored atomic introductions and no primitive zero-source transport.

One inherited negative control changed only the retired introduction-mode field; the compact profiles now correctly accept its provable administrative closure goal, while the strict profile rejects that noncanonical public formula. This expected change is recorded separately, not counted as an unchanged rejection.

The central certificate tests the part most at risk of circularity: recovering a zero-source equality consequence after the two primitive restrictions have been imposed. The named proof is audited for every atomic introduction and every transport source before serialization. A separate audit also traverses every formula goal and transport motive, including nested binders, rather than inspecting only introduction nodes. The accepted expansion uses only mirrored atomic matrices and no primitive zero-source transport. The direct forbidden transport certificate is separately rejected.

Additional cases exercise both computation equations, padding, shared-left implication, nonvacuous witnesses, compound case introduction, induction with a used hypothesis, and public grammar validation. Negative controls cover context and eigenparameter escape, malformed packets, false goals, and forbidden compound progress. These checks support the implementation claims; they are not substitutes for the universal proofs in the preceding sections.

6.6 The smallest byte profile in full

```
import json

def c(a, g=[[]], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 == [] else a if k <= a < d else d %
     2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = m == ([s, u] if s else [t] * 3) and i(t, u) == a and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a := i(*a, t,
     2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or not len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

7 Assessment and limits

The improvement has three separate parts. The atomic public guard uses one iterator with its initializer repeated on the left. The introduction rule is determined by the matrix root. Source-selected elimination retains the previous source restriction. The compatibility proof, especially shared-left implication and the use of $J(y) \to E(y)$ rather than $J(y) \to E(Sy)$ during the induction bootstrap, is what permits the last two reductions to coexist in a canonical theory with the required interpretations.

The earlier two-iterator guarded proposal already eliminated the introduction choice. The earlier source-dispatch proposal already eliminated literal-zero transport sources. Neither result alone supplies the new compatibility theorem; the inert-hypothesis obstruction explains the missing step. The single-sided orientation, its safe expansion argument, the substitution-free induction recovery, and the compiled canonical bootstrap are the contributions of this package. The scalar

arithmetic compiler, logical macros, arity codec, certified-zero computation repair, and most of the zero-symmetry construction are inherited and credited.

The measured compact frontier is strictly improved, but by a small amount. This is not a claim of an absolute simplest axiomatization: without fixing a representation language, allowed encodings, and trusted runtime, a global minimum is not even a stable comparison. Nor does a smaller compact kernel show that a strict input validator or the shortest explanatory proof is smaller. The public grammar restriction and the separate strict profile make those costs visible.

The numerical domain was deliberately retained. A change to tuples, functions, or another coding domain is not needed for this reduction; keeping actual equality makes the two identity composites explicit. No extra induction strength, reflection principle, or universal evaluator was introduced. The next substantive verification step would be the Lean extension described in Section 5, whose missing status is stated rather than implied by the executable Python tests.

A Reference boundary and reproduced arithmetic

The following five sections are inherited mathematical sources from the pinned repository, reproduced unchanged in substance and retained under its MIT-0 licence. They are not claimed as new results of this report. Their references to an "old" calculus or to the novelty of a choice rule describe their original context. Our new result is the mirror-guard calculus and its compatibility proof in the main text. These sources make the arithmetic and ordinary-logic dependencies locally available; the independent new theorem is a written proof, not a Lean build.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent

from F and H is the fixed computation theorem $I(0,0;u.0)=0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use}\quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s=t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t,\eta_\varepsilon(z)) \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z)=0, \quad \eta_1(z)=z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0,1\}$, and $\varepsilon=1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon=0$ the *case choice* and $\varepsilon=1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad\Longleftrightarrow\quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y} E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A,B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A,B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0,s,t) = C(1,s,t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y} B)^\circ = \exists x \big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big),$$

$$\langle A; y.B \rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two

equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0,y) = 1, \qquad L(Sx,0) = 0, \qquad L(Sx,Sy) = L(x,y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx,Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x,y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su,m), u, Su), \qquad W(t,m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su,m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0,m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t,m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t,m)) \\ &= SB_m(W(t,m)) = SW(St,m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n,n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I(m, 0; u.C(L(SSu, m), u, Su)). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I(t, 0; u.C(L(SSu, Sm), u, Su)).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [7]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S((Sx) + y) = S(S(x + y)) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation

induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [8]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] V. Reshetnikov and ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. ProveIt, commit `1517d9d79ce33f631e0f82960c849427cbba23bf`. Pinned repository snapshot.

[2] ProveIt contributors. *Guarded core with a deterministic introduction rule*. `Research/guarded_core`, in [1]; especially `sections/guarded_core.tex` and `sections/implementation.tex`.

[3] ProveIt contributors. *Source dispatch and literal-zero source admissibility.* `Research/source_dispatch`, in [1]; especially `proofs/zero_source_admissibility.tex`.

[4] ProveIt contributors. *Complementary introductions with one local assumption.* `Research/certified_zero/reference/inherited_choice_basis.tex`, in [1].

[5] ProveIt contributors. *NativeBiInterpretation.lean*, *BiInterpretation.lean*, and their transitive imports. `Lean/ConciseIPRA`, in [1].

[6] ProveIt contributors. *Internal scalar predecessor bootstrap.* Frozen reference `Research/certified_zero/reference/scalar_predecessor.tex`, in [1]. The historical attribution key is retained for the reproduced appendix.

[7] ProveIt contributors. *From literal-equation induction to full intuitionistic PRA.* Frozen reference `Research/certified_zero/reference/equality_reflection.tex`, in [1]. The historical attribution key is retained for the reproduced appendix.

[8] ProveIt contributors. *Pairing, primitive recursion, and internal numerical composites.* Frozen reference `Research/certified_zero/reference/primitive_recursion.tex`, in [1]. The historical attribution key is retained for the reproduced appendix.